FSTG: Few-Shot Test Case Generation from Use Case Specifications Using Large Language Models

Thanh-Binh Trinh[1(✉)], Thi-Van Nguyen[1], Ngoc-Minh Le[2], and Nguyen Viet Ha[3]

[1] Faculty of Information Systems, Phenikaa University, Hanoi, Vietnam
binh.trinhthanh@phenikaa-uni.edu.vn
[2] Faculty of Information Technology, Haiphong University, Haiphong, Vietnam
[3] Institute for Artificial Intelligence, VNU University of Engineering and Technology, Hanoi, Vietnam

Abstract. Automated test case generation from requirements specifications remains challenging, especially in DevOps workflows where continuous validation demands fast and reliable test creation. This paper proposes FSTG, a few-shot approach for generating test cases from Cockburn-style use cases using large language models. FSTG extracts semantic information from use cases and leverages schema-guided prompting to produce semantically consistent test cases with only a small number of demonstrations. To assess the effectiveness of the generated tests, the proposed approach includes an automated evaluation pipeline for measuring coverage and redundancy. Experiments on 112 use cases from five real-world systems show that FSTG achieves 89.2% transition coverage and detects 31.2 faults on average, showing improvements over both model-based and zero-shot baselines while reducing test generation time. These results indicate that FSTG reduces the gap between informal requirements and automated testing, enabling practical integration of AI-assisted test generation into DevOps workflows.

Keywords: Few-shot · Test case generation · Use case specifications · Large language models · DevOps

1 Introduction

Use case specifications serve as fundamental artifacts in requirements engineering, capturing functional requirements from a user perspective through well-defined scenarios of system interactions [1]. In practice, use cases are typically specified in Cockburn's style, which organizes the scenario into preconditions, postconditions, main success scenario, extensions, and invariants. This structure supports systematic test derivation in DevOps workflows, where continuous validation depends on timely and consistent test creation [2]. However, the manual translation of use cases into comprehensive test suites remains time-consuming

N. T. Nguyen et al. (Eds.): ICCIES 2026, CCIS 2945, pp. 302–316, 2026.
https://doi.org/10.1007/978-3-032-21631-1_22

and requires significant testing expertise [3]. This limitation often results in insufficient test coverage, slower feedback cycles, and reduced testing effectiveness in DevOps workflows. To address this challenge, several automated test case generation approaches have been developed, which can be classified into two categories: model-based and AI-driven methods.

Model-based approaches, such as TCG [4], transform Cockburn-style use case specifications into formal representations like control flow graphs and apply depth-first search (DFS) to generate test paths, which are then optimized using genetic algorithms (GA). While effective, these approaches often require substantial formalization effort, strict adherence to specific templates, and may encounter difficulties with variations in natural language descriptions. In contrast, AI-driven techniques leveraging Large Language Models (LLMs) can handle less formalized use cases, but typically require extensive fine-tuning on large datasets [5] or may produce inconsistent results in zero-shot scenarios [6].

Few-shot prompting offers a promising approach to address the limitations of both model-based and fully fine-tuned AI methods [7]. In few-shot prompting, a model can adapt to new tasks using only a small number of examples, without requiring extensive retraining. This capability mirrors how human testers operate—learning testing patterns from limited demonstrations and applying them to new scenarios. Despite its potential, the use of few-shot prompting for generating test cases from Cockburn-style use case specifications remains largely unexplored, particularly in the context of DevOps, where maintaining traceability between requirements and tests is critical.

In this paper, we propose `FSTG` (Few-shot test case generation), a novel framework that leverages few-shot prompting to automate test case generation from use case specifications. Our approach requires no model retraining or extensive formalization, instead using carefully selected examples to guide LLMs in generating comprehensive test suites. The generated test cases are systematically evaluated for coverage of the specified scenarios and for semantic validity, ensuring that they not only exercise the intended functionality but also conform to the constraints and conditions defined in the original use cases. The main contributions of this work are:

- A unified framework (FSTG) for few-shot test case generation from Cockburn-style use case specifications, integrating semantic extraction, schema-constrained prompting, and in-context learning.
- A structured prompt engineering methodology that systematically transforms narrative use cases into LLM-ready representations.
- An empirical evaluation on 112 use cases from five real-world systems, showing that FSTG delivers higher coverage, better fault detection, and faster generation than existing baselines, while integrating effectively into DevOps workflows.

The rest of the paper is organized as follows. Section 2 provides a brief overview of related work. Our `FSTG` approach for automated test case generation from use case specifications is presented in Sect. 3. Experimental results

and discussion are provided in Sect. 4. Finally, Sect. 5 concludes the paper and outlines directions for future work.

2 Related Work

Test case generation has long been considered a core activity in software engineering to ensure software quality and reliability, encompassing both code-based and specification-based approaches. Recent advances in automated test case generation [5,6,8,9,16] have explored a wide range of techniques, including model-based methods, evolutionary algorithms, reinforcement learning, and large language models (LLMs). These approaches are particularly relevant in DevOps workflows, where early-stage automation from requirements is essential to shorten feedback cycles and improve testing effectiveness.

Specification-Based and Model-Based Test Generation. One established research direction focuses on deriving test cases from software specifications such as use case descriptions or UML models. Wang et al. [8] propose an automated approach that transforms Cockburn-style use cases into control-flow representations and applies genetic algorithms to optimize test coverage. Similarly, Sahoo et al. [16] introduce a UML-driven method that integrates use case, activity, and sequence diagrams into system graphs for evolutionary test generation. While effective, these approaches typically require substantial formalization effort and strict conformance to modeling templates, which can limit their applicability in fast-evolving DevOps workflows.

AI-Driven and Learning-Based Testing. More recent work leverages artificial intelligence to provide adaptive and data-driven testing mechanisms. PyTester [9] is a representative example, employing deep reinforcement learning to generate syntactically correct and executable test cases directly from natural-language requirements. By optimizing reward functions that capture syntax validity, executability, and code coverage, PyTester outperforms strong LLM baselines such as GPT-4.0 [10] and StarCoder [17]. In parallel, comprehensive tertiary studies [18] highlight the increasing adoption of evolutionary algorithms, machine learning models, and hybrid AI techniques in software testing. However, most AI-driven approaches primarily target system-level or code-level testing and provide limited support for explicit traceability to early-stage requirement artifacts such as use case specifications.

LLM-Based and Prompting-Driven Test Generation. The emergence of large language models has significantly influenced software testing research, enabling new forms of test case generation and automated validation. Recent surveys report the growing role of LLMs in software testing tasks, including test generation, test maintenance, and oracle construction [13]. Prompt-based learning, particularly few-shot and in-context prompting, has emerged as a promising paradigm. Brown et al. [7] demonstrate that LLMs can adapt to new tasks using

only a small number of examples, while systematic surveys and prompt pattern catalogs [20] further analyze how prompt engineering techniques can guide and stabilize LLM behavior in engineering tasks.

Despite these advances, existing LLM-based testing approaches largely rely on extensive fine-tuning [5], which incurs significant data and computational costs, or exhibit unstable behavior under zero-shot prompting [6]. Moreover, few-shot prompting techniques specifically tailored to generating test cases from structured use case specifications remain largely underexplored, particularly with respect to ensuring behavioral coverage, output consistency, and DevOps workflows.

Summary and Positioning. These observations highlight the need for approaches that bridge informal or semi-structured requirements and automated testing, while preserving traceability, behavioral completeness, and alignment with DevOps workflows. In contrast to prior work, our approach focuses on a systematic integration of semantic extraction, schema-constrained prompting, and few-shot demonstrations to generate test cases directly from Cockburn-style use case specifications, without requiring model retraining or heavy formalization.

3 FSTG: Few-Shot Test Case Generation Framework

In this section, we propose a few-shot test case generation framework based on large language models, which systematically derives test cases from use case specifications through semantic extraction, prompt synthesis, and few-shot demonstrations.

3.1 Overall FSTG Framework

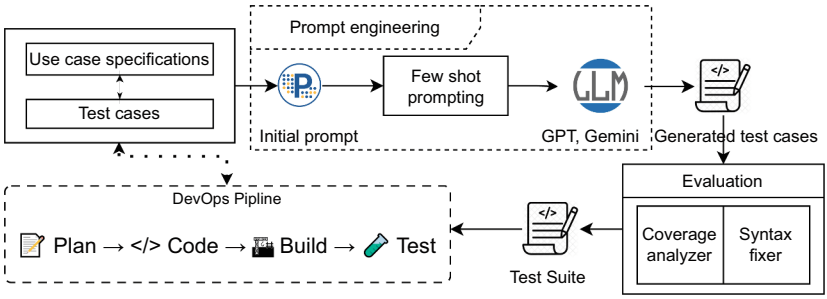

Fig. 1. Overall framework for few-shot test case generation from use case specifications, incorporating prompt construction, LLM-based synthesis, automated evaluation, and integration into a DevSecOps workflows.

Figure 1 illustrates the overall framework for few-shot test case generation from use case specifications. The framework comprises four tightly connected

components: (i) structured prompt construction, where the use case is transformed into an initial prompt enriched with few-shot demonstrations; (ii) LLM-based test case synthesis, in which a large language model generates candidate test cases that capture the intended behavioral semantics; (iii) automated evaluation and refinement, including semantic coverage analysis and syntax normalization to produce a consistent and executable test suite; and (iv) integration into a DevOps workflows, enabling continuous and automated validation within modern software delivery workflows. Together, these components form an end-to-end process for converting Cockburn-style use case descriptions into high-quality, semantically grounded test cases.

3.2 Prompt Synthesis

Prompt synthesis is a foundational component of the FSTG framework, as it determines how the large language model (LLM) interprets the use case specification and synthesizes semantically valid test cases. The objective of this stage is to construct a structured and well-scoped prompt that accurately conveys the testing goals while preserving the behavioral semantics encoded in the original requirements. To achieve this, FSTG employs a three-step strategy: (i) semantic extraction from the use case, (ii) structured prompt construction, and (iii) integration of few-shot demonstrations.

Algorithm 1. Semantic extraction

Require: Use case specification UC in Cockburn style (Goal, Preconditions, Main-Flow, Extensions)
Ensure: Normalized semantic model $\{\text{Preconds}, \text{MF}, \text{EXT}, \text{Paths}\}$

```
Initialize Preconds ← ∅, MF ← [], EXT ← ∅, Paths ← []
Preconds ← normalize(UC.Preconditions)
MF ← normalizeSteps(UC.MainFlow)
for each extension ext in UC.Extensions do
    (step, cond, alt) ← parseExtension(ext)
    EXT ← EXT ∪ {(step, cond → alt)}
end for
Paths.append(MF)                                   ▷ main success path
for each (step, cond → alt) in EXT do
    branch ← copy(MF)
    replaceStep(branch, step, alt)
    Paths.append(branch)
end for
return {Preconds, MF, EXT, Paths}
```

Semantic Extraction. Use cases written in Cockburn's style capture key behavioral elements such as goals, preconditions, main flows, and extensions, but their narrative form may hinder automated reasoning. We therefore propose

a semantic extraction approach (Algorithm 1) that normalizes preconditions and main flows, parses extensions to identify conditional branches, and enumerates all executable paths. The resulting structured, machine-interpretable model makes causal relations and decision points explicit, providing the basis for subsequent prompt construction. This step is implemented in Python using simple rule-based parsing and standard data structures.

Table 1. Use case specification: process order

Element	Description
Name	Process Order
Goal	Handle the end-to-end processing of a customer order.
Preconditions	Items are available in the shopping cart; a valid payment method is provided.
Main Success Scenario	1. The customer submits the order. 2. The system validates and processes the payment. 3. The system updates the inventory. 4. The system issues an order confirmation.
Extensions	(2a) Payment processing fails → The system cancels the order. (3a) Inventory is insufficient → The system cancels the order.

As an example, consider the Cockburn-style use case in Table 1, which has a main success scenario and two extension conditions. Applying the proposed semantic extraction algorithm (Algorithm 1) produces a structured model with main-flow steps $MF = \langle MF_1, MF_2, MF_3, MF_4 \rangle$ and extensions $EXT = \{EX_{2a}, EX_{3a}\}$, where each extension is a pair $(MF_i, cond \rightarrow alt)$. Formally, the extracted structure is:

$$
\begin{aligned}
&Preconds = \{ItemsAvailable, PaymentValid\},\\
&MF_1 : SubmitOrder,\\
&MF_2 : ProcessPayment,\\
&MF_3 : UpdateInventory,\\
&MF_4 : ConfirmOrder,\\
&EX_{2a} : (MF_2,\ PaymentFailed \rightarrow CancelOrder),\\
&EX_{3a} : (MF_3,\ InsufficientInventory \rightarrow CancelOrder).
\end{aligned}
$$

Based on these components, the algorithm enumerates all executable paths:

$$
\begin{aligned}
&\pi_{\text{main}} = \langle MF_1, MF_2, MF_3, MF_4 \rangle,\\
&\pi_{2a} = \langle MF_1, MF_2, CancelOrder \rangle,\\
&\pi_{3a} = \langle MF_1, MF_2, MF_3, CancelOrder \rangle.
\end{aligned}
$$

Structured Prompt Construction. Building on the normalized behavioral model obtained from the semantic extraction phase, we construct the initial prompt for LLM-driven test synthesis using a reusable prompt template.

The template in Listing 1.1 is organized into four logical sections. The *Task* section states the test generation objective and constrains the LLM to operate strictly within the semantics of the given use case. The *Coverage* section enumerates the executable paths obtained from semantic extraction, directly reflecting the *Paths* output of Algorithm 1. The *Output Format* section enforces a fixed test case schema, which enables deterministic post-processing and normalization of generated test cases. Finally, the *Use Case Model* section injects the normalized preconditions, main-flow steps, and extension rules, providing the factual grounding required for faithful test synthesis.

This template embeds the testing objective, enforces a fixed output schema, and encodes behavioral coverage requirements derived from the extracted execution paths. By instantiating this template with a specific normalized use case model, FSTG ensures that the LLM receives a semantically grounded, schema-constrained, and coverage-complete prompt.

```
=== TASK ===
Generate test cases that preserve the use case semantics
    and cover all executable paths.
=== COVERAGE ===
Paths to be covered:
<PATH_1>, <PATH_2>, ..., <PATH_n>
=== OUTPUT FORMAT ===
Test ID:
Preconditions:
Inputs:
Execution Steps:
Expected Outcome:
Covered Flow:
=== USE CASE MODEL ===
Preconditions: <PRECOND_1>, <PRECOND_2>, ...
Main Flow:
MF1: <ACTION_1>
MF2: <ACTION_2>
...
Extensions:
EXi: <CONDITION_i> -> <ALTERNATIVE_i>
```

Listing 1.1. Generic structured prompt template for FSTG

Few-Shot Demonstrations. Building on the structured prompt constructed in the previous step, FSTG further guides the LLM by embedding a small number of few-shot demonstrations that illustrate how a normalized behavioral model is transformed into executable, schema-compliant test cases. Rather than providing ad hoc examples, FSTG adopts a generic few-shot demonstration template that is instantiated with representative execution paths derived from the semantic model.

Each few-shot example corresponds to exactly one executable path (either the main success path or an extension-induced branch) and realizes this path as

a complete test case following the predefined schema. The demonstrations therefore make explicit the mapping between formal execution paths and concrete test artifacts, reinforcing the intended reasoning pattern of the LLM.

Listing 1.2 shows the generic structure of few-shot demonstrations used in FSTG. Each example specifies the preconditions, inputs, execution steps, and expected outcome, and explicitly records the covered execution path via the *Covered Flow* field. This explicit linkage ensures that the LLM learns to associate each generated test case with a valid execution path extracted from the use case.

```
=== FEW-SHOT DEMONSTRATIONS ===
Example <k>
Test ID: <ID_k>
Preconditions: <PRECOND_ASSIGNMENTS>
Inputs: <INPUT_VALUES>
Execution Steps:
  1. <STEP_1>
  2. <STEP_2>
  ...
Expected Outcome: <EXPECTED_RESULT>
Covered Flow: <PATH_ID_k>
```

Listing 1.2. Generic few-shot demonstration template for FSTG

The selection of few-shot demonstrations follows three simple principles derived from the semantic model: (i) the examples collectively cover at least one main-flow path and one extension path; (ii) they exhibit variation in triggering conditions and outcomes; and (iii) they strictly conform to the output schema enforced by the structured prompt. This controlled use of few-shot examples constrains the LLM's generative behavior without requiring parameter updates, stabilizes output structure, and ensures that each synthesized test case remains semantically aligned with the underlying use case model.

In FSTG, few-shot demonstrations are fixed with respect to the use case specification format rather than dynamically adapted per instance. For each common use case structure (e.g., main flow with extensions), a small set of representative examples is curated once and reused across all use cases sharing the same pattern, ensuring prompt stability and reproducibility.

Illustrative Prompt. In the final stage, FSTG deterministically assembles the normalized use case model, the fixed test case schema, and the selected few-shot demonstrations into a single unified prompt. This assembled prompt provides the LLM with an explicit specification of *what* to generate (testing objective), *how* to generate it (schema constraints), and *which behaviors* must be covered (execution paths). By construction, the resulting prompt is systematic, reusable across different use cases, and preserves the behavioral semantics extracted from the original requirements.

3.3 Evaluation and DevOps Integration

LLM-based synthesis generates candidate test cases that may contain syntactic inconsistencies or coverage gaps. FSTG therefore includes an evaluation stage—syntax normalization, semantic coverage analysis, and DevOps integration—that acts as a quality gate before downstream testing (Fig. 1).

Syntax and Structural Normalization. Even under a schema-constrained prompt, LLM-generated test cases may exhibit minor structural deviations, such as missing fields, inconsistent ordering, or loosely formatted execution steps. While these deviations are typically tolerable for human readers, they can hinder automated parsing, execution, and coverage analysis. To address this issue, the evaluation stage begins with a syntax normalizer that enforces strict compliance with the test-case schema introduced during structured prompt construction (Sect. 3.1).

Given a generated test case T and a schema

$$\mathcal{S} = \{s_1, \ldots, s_m\},$$

the normalizer reconstructs a canonical representation T^* by ensuring: (i) complete inclusion of all schema fields, (ii) uniform formatting of structured content, and (iii) preservation of flow identifiers required for subsequent semantic coverage analysis.

Algorithm 2. Schema-based syntax normalization

Require: Generated test case T, schema $\mathcal{S} = \{s_1, \ldots, s_m\}$
Ensure: Normalized test case T^*
Initialize T^* as an empty structured object
for each field s_i in $\mathcal{S}$ **do**
  if s_i appears in T **then**
    $v \leftarrow extract(T, s_i)$
    $v \leftarrow normalize\_format(v)$
    append (s_i, v) to T^*
  else
    append $(s_i, default\_value(s_i))$ to T^*
  end if
end for
reorder fields of T^* according to $\mathcal{S}$
return T^*

This normalization step ensures that all downstream components operate on structurally consistent and machine-processable test artifacts that conform exactly to the schema enforced during prompt construction (Sect. 3.1), thereby enabling reliable semantic coverage analysis and seamless integration into subsequent testing activities.

Semantic Coverage Analysis. Beyond structural correctness, the generated test suite must achieve complete behavioral coverage of the use case. The normalized semantic model obtained during extraction,

$$\mathcal{U} = \langle \text{Name}, \text{Pre}, \mathcal{F}_{\text{main}}, \mathcal{F}_{\text{ext}} \rangle,$$

serves as the reference for coverage assessment.

Each normalized test case T_i includes a *CoveredFlow* field indicating the behavioral branch it exercises. Based on this information, the coverage analyzer evaluates the test suite with respect to: (i) completeness over all required flows $\mathcal{F}_{\text{main}} \cup \mathcal{F}_{\text{ext}}$, (ii) branch-level adequacy, and (iii) redundancy among semantically equivalent test cases.

Algorithm 3. SemanticCoverageAnalyzer

Require: Semantic model $\mathcal{U} = \langle Name, Pre, F_{\text{main}}, F_{\text{ext}} \rangle$
Normalized test suite $T = \{T_1, \ldots, T_n\}$
Ensure: Coverage report R
$F_{\text{req}} \leftarrow F_{\text{main}} \cup F_{\text{ext}}$
$F_{\text{cov}} \leftarrow \emptyset$, $Duplicates \leftarrow \emptyset$
for each test case T_i in T **do**
    $F_i \leftarrow extract\_covered\_flows(T_i)$
    $F_{\text{cov}} \leftarrow F_{\text{cov}} \cup F_i$
    if T_i is semantically equivalent to any prior test case **then**
        record duplicate in *Duplicates*
    end if
end for
$Missing \leftarrow F_{\text{req}} \setminus F_{\text{cov}}$
return $R \leftarrow build\_report(F_{\text{req}}, F_{\text{cov}}, Missing, Duplicates)$

The resulting report summarizes achieved coverage, identifies missing behavioral branches, and highlights redundant test cases. This information enables systematic validation of LLM-generated tests before their integration into subsequent testing activities.

DevOps Integration. After normalization and semantic validation, the generated test suite is integrated into the *Test* stage of the DevOps workflows. This integration enables continuous verification of use casedriven behaviors.

Validated test cases are deployed to the target testing framework (e.g., PyTest, JUnit, or Postman/Newman). When use cases evolve, the pipeline reinvokes the FSTG workflow—from semantic extraction to evaluation—ensuring that the test suite remains aligned with the updated requirements. Coverage-based quality gates prevent pipeline progression if required behavioral flows are missing or schema violations persist. In addition, explicit traceability links between test cases and semantic elements of the use case model $\mathcal{U}$ support maintainability and auditing.

By embedding evaluation results directly into the pipeline, FSTG operates as a systematic and reproducible testing component within DevOps, ensuring that generated test artifacts remain structurally valid, behaviorally complete, and continuously aligned with evolving requirements.

4 Experimental Result

To validate the proposed approach, we conducted an experimental evaluation on five operational systems from the e-commerce, education, finance, and healthcare domains, comprising 112 use cases with approximately 320 executable flows. Three baselines were considered for comparison: manual test design, model-based test case generation, and zero-shot LLM-based generation. All test suites were normalized and evaluated in terms of transition coverage, fault detection capability, generation time, and redundancy ratio.

4.1 Result

Table 2 summarizes the transition coverage achieved across all systems. FSTG (with three demonstrations) attains 89.2% coverage, approaching the 92.8% achieved by manual test design and substantially exceeding both TCG (71.8%) and zero-shot prompting (47.6%). The results indicate that the combination of semantic extraction, schema constraints, and few-shot prompting enables FSTG to systematically identify both main and alternative flows. The largest relative improvement occurs in use cases containing multiple extension branches, where baselines often fail to capture alternative paths not explicitly emphasized in the narrative structure.

Table 2. Transition coverage across all systems

Method	Coverage
Manual testing	92.8%
FSTG (k=3)	89.2%
TCG	71.8%
Zero-shot LLM	47.6%

Fault detection performance is summarized in Table 3. FSTG identifies on average 31.2 injected faults, which is close to manual testing (34.7) and clearly higher than both TCG (27.1) and zero-shot prompting (18.3). These results demonstrate that the semantic fidelity of the generated tests—grounded in the extracted behavioral structure and regulated by schema compliance—has a direct impact on their functional relevance. In particular, extension flows that are frequently omitted by the baselines account for a substantial proportion

Table 3. Fault detection results

Method	Faults Detected (avg.)
Manual testing	34.7
FSTG (k=3)	31.2
TCG	27.1
Zero-shot LLM	18.3

of the detected faults, reinforcing the importance of capturing full behavioral variability.

Table 4 shows the generation time required for each method. Manual test construction ranges from six to eighteen hours, depending on use case complexity. TCG requires between twelve and forty-eight minutes due to control-flow graph construction and optimization. By contrast, FSTG completes generation in 3.2 s on average—marginally slower than zero-shot prompting (2.8 s) but effectively instantaneous relative to the baselines. The small overhead arises primarily from semantic extraction and redundancy analysis, both of which remain lightweight. These results confirm that the framework is suitable for integration into fast-paced DevSecOps pipelines where rapid feedback is critical.

Table 4. Test generation time

Method	Time
Manual testing	6–18 h
TCG	12–48 min
FSTG (k=3)	3.2 s
Zero-shot LLM	2.8 s

To evaluate the impact of demonstrations, we vary k from 0 to 5 and measure coverage and redundancy. Table 5 shows that zero-shot prompting results in low coverage and high redundancy, while even one demonstration improves consistency. Performance increases up to $k = 3$, after which coverage reaches a limit and redundancy slightly rises, indicating that three demonstrations provide an optimal balance between coverage and output reliability.

4.2 Discussion

The experimental results show that FSTG consistently outperforms zero-shot prompting and conventional model-based approaches across all evaluated metrics. By combining semantic extraction with schema-constrained prompting and few-shot demonstrations, FSTG guides large language models to systematically

Table 5. Ablation results for different values of k

k	Coverage	Redundancy	Notes
0	47.6%	High	Few flows identified
1	72.4%	Moderate	Partial stabilization
2	84.1%	Low	Consistent structure emerges
3	89.2%	Low	Best trade-off
5	89.4%	Higher	Diminishing returns

explore both main and alternative flows, leading to higher transition coverage and fault-detection performance while preserving near-instantaneous generation time.

The ablation analysis confirms the importance of few-shot demonstrations in stabilizing output structure and improving coverage. A small number of curated examples is sufficient to achieve optimal performance, while additional demonstrations provide diminishing returns. Despite these advantages, FSTG currently relies on rule-based semantic extraction and assumes Cockburn-style use case templates, which may limit its effectiveness for highly complex or irregular specifications. Moreover, the evaluation focuses on requirement-level coverage and does not yet consider automated oracle generation or execution-level feedback.

To synthesize these findings from a practical perspective, Table 6 presents a qualitative comparison of the evaluated approaches derived from the quantitative results and empirical observations. Specifically, coverage and scalability reflect the measured transition coverage and behavior across 112 use cases, while formalization effort and DevOps suitability are inferred from observed modeling overhead, prompt stability, and integration complexity.

Table 6. Qualitative comparison derived from empirical observations

Method	Formalization effort	Coverage	Scalability	DevOps suitability
Manual testing	High	High	Low	Medium
TCG (model-based)	High	Medium	Medium	Low
Zero-shot LLM	Low	Low	High	Medium
FSTG (proposed)	Low	High	High	High

Overall, the results indicate that FSTG strikes a favorable balance between automation, coverage, and practical deployability, making it well suited for use casedriven test generation in modern DevOps workflows.

5 Conclusion

In this paper, we proposed FSTG, a few-shot framework for generating test cases from Cockburn-style use case specifications. By integrating semantic extrac-

tion, schema-constrained prompts, and demonstration-based in-context learning, FSTG enables large language models to produce behaviorally faithful and structurally consistent test suites with minimal human intervention.

An evaluation on 112 use cases from five real-world systems shows that FSTG achieves coverage and fault-detection comparable to manual test design while drastically reducing generation time. Compared to model-based and zero-shot approaches, FSTG provides higher semantic completeness, lower redundancy, and more stable output structure, requiring only a few curated demonstrations to generalize across domains.

These results position few-shot prompting as an effective bridge from informal requirements to automated testing, with future work exploring richer requirement notations, automated oracles, and large-scale system testing.

References

1. Cockburn, A.: Writing Effective Use Cases, 1st edn. Addison-Wesley, USA (2000)
2. Beck, K.: Test-Driven Development: By Example. Addison-Wesley, Boston, MA (2002)
3. Ammann, P., Offutt, J.: Introduction to Software Testing. Cambridge University Press, Cambridge (2016)
4. Trinh, T.-B., Truong, N.-T.: TCG: An Approach to Improving Test-driven Development by Exploiting Use Case Specifications. Int. J. Syst. Assur. Eng, Manag (2025)
5. Shang, Y., Zhang, Q., Fang, C., Gu, S., Zhou, J., Chen, Z.: A large-scale empirical study on fine-tuning large language models for unit testing. Proc. ACM Softw. Eng. **2**(ISSTA), Article ISSTA074 (2025)
6. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Trans. Softw. Eng. **50**(4), 911–936 (2024)
7. Brown, T.B., et al.: Language models are few-shot learners. In: Proceeding 34th Conference Neural Information Processing Systems (NeurIPS), pp. 1–12 (2020)
8. Wang, C., Pastore, F., Goknil, A., Briand, L., Iqbal, Z.: Automatic generation of system test cases from use case specifications. In: Proc. Int. Symp. Software Testing and Analysis (ISSTA 2015), pp. 385–396. ACM, New York (2015)
9. Zhong, H., Li, X., Wu, J.: PyTester: deep reinforcement learning for automatic test case generation from natural language requirements. In: Proceeding International Conference Software Engineering (ICSE), pp. 1–12 (2024)
10. OpenAI: GPT-4 Technical Report (2023)
11. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G.: Pre-train, prompt, and predict: a systematic survey of prompting methods in natural language processing. ACM Comput. Surv. **55**(9), Article 195 (2023)
12. Reynolds, L., McDonell, K.: Prompt programming for large language models: beyond the few-shot paradigm. In: Extended Abstracts of the CHI Conf. Human Factors in Computing Systems (CHI EA 2021), Article 314, pp. 1–7. ACM, New York (2021)
13. Hou, X., et al.: Large language models for software engineering: a systematic literature review. ACM Trans. Softw. Eng. Methodol. **33**(8), Article 220 (2024)

14. Utting, M., Legeard, B.: Practical model-based testing: a tools approach. Morgan Kaufmann, San Francisco, CA (2006)
15. Shahin, M., Babar, M. A., Zhu, L.: Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. CoRR abs/1703.07019 (2017)
16. Sahoo, R.K., Derbali, M., Jerbi, H., Thang, D.V., Kumar, P.P., Sahoo, S.: Test case generation from UML diagrams using genetic algorithm. Comput. Mater. Continua **67**(2), 2321–2336 (2021)
17. Li, R., et al.: StarCoder: May the source be with you! CoRR abs/2305.06161 (2023)
18. Amalfitano, D., Faralli, S., Hauck, J. C. R., Matalonga, S., Distante, D.: Artificial intelligence applied to software testing: A tertiary study. ACM Comput. Surv. **56**(3), Article 58 (2023)
19. Olsthoorn, M.: More effective test case generation with multiple tribes of AI. In: Proc. ACM/IEEE 44th Int. Conf. Software Engineering (ICSE Companion), pp. 286–290. ACM, New York (2022)
20. White, J., et al.: A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT, In: Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP 2023), Article 5, pp. 1–31, 2023